UJIAN TENGAH SEMESTER — KOMPUTASI PARALEL DAN TERDISTRIBUSI

ANT WORLD

Hybrid Parallel Ant Colony Simulation

Hybrid Computing for Real-World Simulation and Data Processing

Ripki Maulana · NIM 247006111139

Program Studi Informatika · Fakultas Teknik · Universitas Siliwangi · 2026



MASALAH & MOTIVASI

Banyak keputusan agen, diulang tiap iterasi

Agen lokal

Tiap semut memilih langkah hanya dari feromon tetangga dan jarak ke sarang.

Efek kolektif

Perilaku koloni muncul dari interaksi lokal (swarm intelligence), bukan kendali pusat.

Beban berulang

Setiap iterasi menghitung keputusan untuk 1.390 agen, berulang ratusan kali.

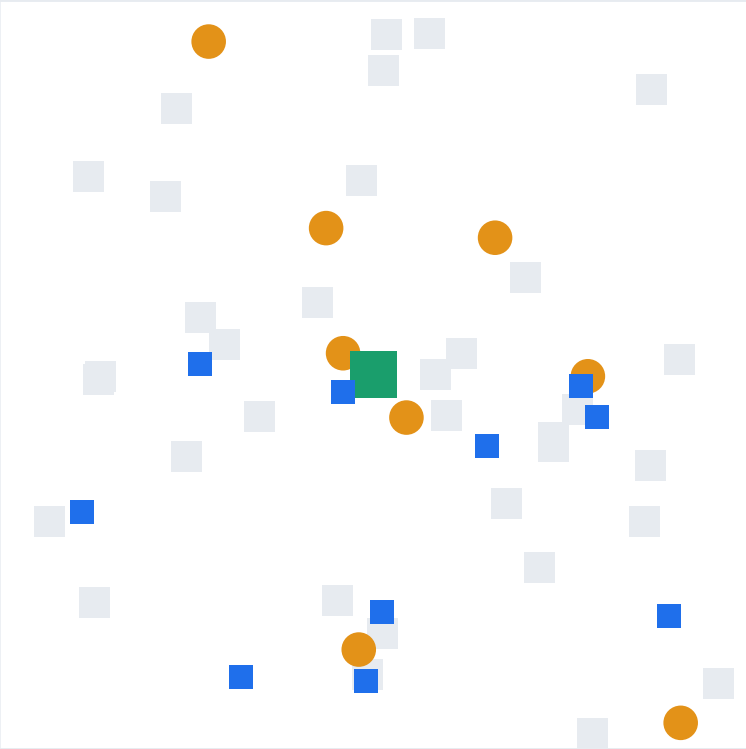
Pertanyaan inti

Bisakah simulasi iteratif ini dipercepat tanpa mengubah hasil?

Menjawabnya = mengevaluasi pendekatan hybrid pada beban nyata.

MENGENAL ANT WORLD

Grid dunia: sarang, semut, makanan, rintangan



Ilustrasi skematik grid 100×100 (bukan hasil pengukuran runtime).

Sarang Semut Makanan Rintangan

Siklus eksplorasi

1. Cari makanan

menjelajah tetangga; feromon + heuristic jarak

2. Ambil makanan

satu unit saat mencapai sumber

3. Kembali ke sarang

heuristik jarak ke nest; simpan path

4. Deposit feromon

rute sukses menandai jalur; feromon menguap

KONFIGURASI EKSPERIMEN

Angka-angka personal dari NIM 24700611139

Dunia

Grid

100 × 100

Sumber makanan

10

Rintangan

20

Sarang

(50, 50)

Paralelisme

Semut

1.390

Thread I/O

5

Worker proses

3

Seed

247006111139

Algoritma

Pheromone decay

0,1

Pheromone deposit

10,0

Alpha / beta

1,0 / 2,0

Iterasi benchmark

100

Parameter personal: thread = 39 mod 4 + 2, proses = 61 mod 3 + 2, data = 139 × 10. Maksimum konfigurasi 1.000 iterasi; benchmark memakai 100.

MENGAPA HYBRID?

Dua jenis beban kerja memerlukan dua mekanisme berbeda

CPU-bound

→ **ProcessPoolExecutor**

Menghitung keputusan & pergerakan semut, yaitu bytecode Python murni.

GIL CPython mencegah thread menjalankan bytecode Python paralel.

Proses punya interpreter & GIL sendiri → paralelisme CPU nyata.

I/O-bound

→ **ThreadPoolExecutor**

Checkpoint JSON, statistik/CSV, dan pembaruan state dashboard.

Tugas lebih banyak menunggu file/jaringan daripada menghitung.

Saat satu thread menunggu, thread lain berjalan; ringan karena berbagi proses.

Bukan klaim "semua lebih cepat": hybrid hanya relevan bila beban CPU cukup besar untuk menutup biaya koordinasi.

ARSITEKTUR SISTEM HYBRID

ProcessPool (CPU) dan ThreadPool (I/O) di bawah satu main process

MAIN PROCESS — World State

World + 1.390 semut · build snapshot grid · bagi ant jadi chunk · merge hasil · update world

JALUR CPU — ProcessPoolExecutor

- Menerima chunk state semut + snapshot world
- Menghitung keputusan/pergerakan semut (bytecode Python)
- Proses terpisah → interpreter & GIL sendiri
- Mengembalikan hasil lokal ke main process

JALUR I/O — ThreadPoolExecutor

- Checkpoint JSON · statistik JSON · CSV benchmark
- Pembaruan state dashboard (ThreadingHTTPServer)
- Dikirim sebagai future; ditunggu saat hasilnya perlu
- Berbagi satu proses → ringan untuk kerja menunggu

Sinkronisasi: snapshot → chunk → submit ke ProcessPool → barrier (tunggu semua future) → urutkan per chunk_id → merge → world update → iterasi berikutnya

MPI tidak digunakan; implementasi berjalan pada satu komputer (tanpa node/rank).

ALUR SATU ITERASI

Dari snapshot hingga world update



Worker menghitung state lokal dari snapshot; tidak menulis world global langsung.

Main process menunggu seluruh future, mengurutkan hasil dengan chunk_id, lalu menerapkan update sekali, secara deterministik.

Iterasi berikutnya dimulai setelah world terbaru tersedia.

Di samping pipeline ini, ThreadPool mengirim tugas I/O sebagai future: dijalankan asinkron, ditunggu hanya saat hasilnya diperlukan.

IMPLEMENTASI & TEKNOLOGI

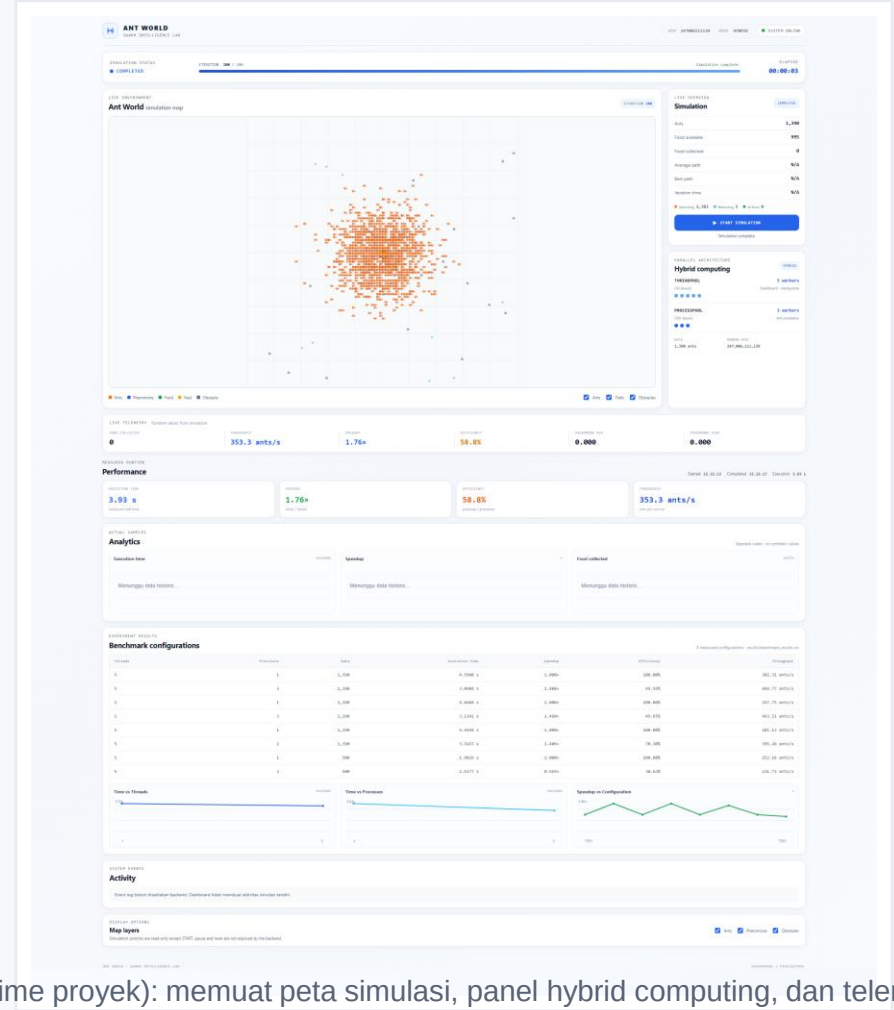
Setiap teknologi dan perannya di proyek

Python 3.14.3	Bahasa implementasi seluruh proyek.
NumPy	Grid world: makanan (int), rintangan (bool), feromon (float); metrik.
concurrent.futures — ProcessPoolExecutor	Worker CPU-bound: gerak & keputusan semut (src/parallel.py).
concurrent.futures — ThreadPoolExecutor	Tugas I/O: checkpoint, statistik, CSV, dashboard (src/io_tasks.py).
http.server — ThreadingHTTPServer	Dashboard lokal + API state, start/pause/resume/restart.
matplotlib / pandas	Grafik benchmark dan tabel hasil (scripts/benchmark.py).

Tidak ada teknologi tanpa peran nyata; daftar ini memetakan pustaka ke fungsinya.

BUKTI APLIKASI

Dashboard ANT WORLD — screenshot asli



Screenshot asli dashboard ANT WORLD (bukti runtime proyek): memuat peta simulasi, panel hybrid computing, dan telemetry.

METODOLOGI BENCHMARK

Setiap konfigurasi diukur berpasangan

Baseline	Eksekusi serial diukur pada workload yang sama sebagai pembanding.
Hybrid	Eksekusi ProcessPool + ThreadPool pada throughput/seed/iterasi identik.
Iterasi	100 iterasi per konfigurasi; seed, jumlah semut, dan iterasi disamakan.
Kondisi	Batch benchmark dijalankan tanpa server dashboard aktif.
Metrik	Execution time, speedup, efisiensi, throughput, dan determinisme hasil.

Perangkat uji

Intel Core i3-4160

2 physical core · 4 logical processor

RAM ± 7,92 GB

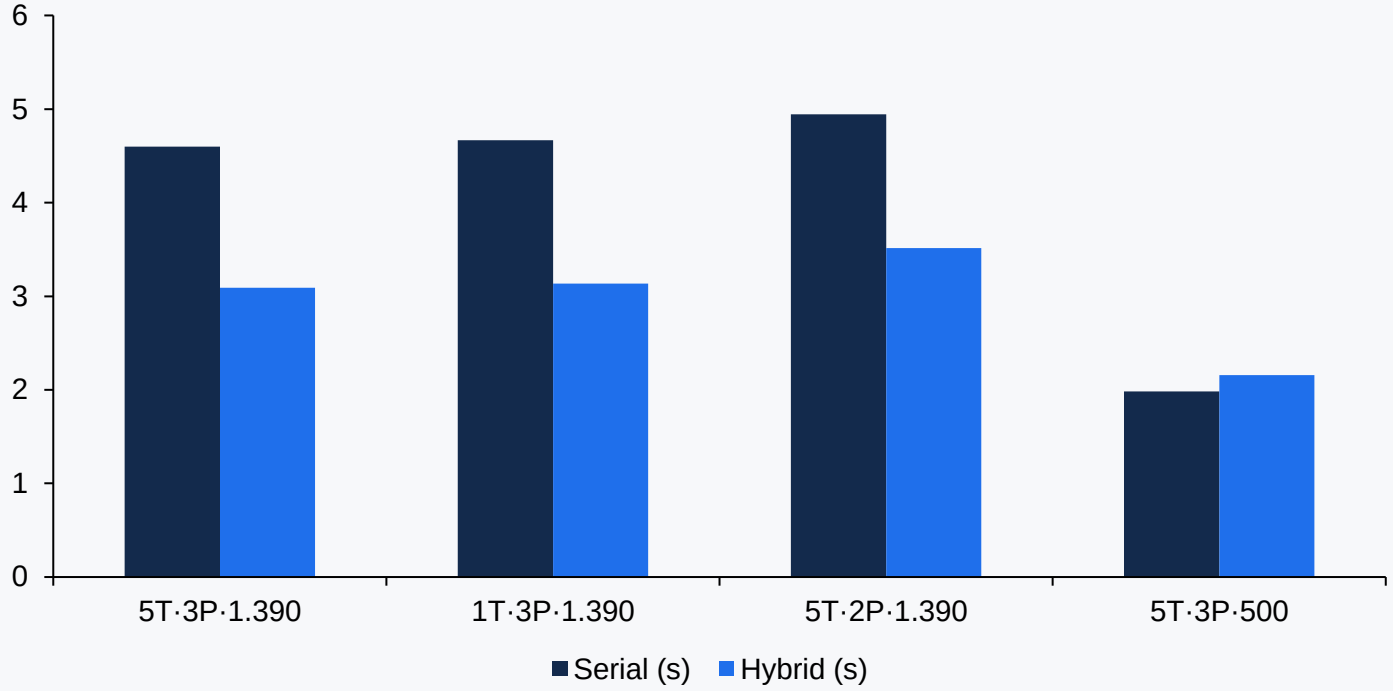
Windows 10 Pro 64-bit

Python 3.14.3

HASIL EKSPERIMEN

Serial vs hybrid per konfigurasi (100 iterasi)

Waktu eksekusi (detik) — label nilai pada batang



Speedup (T_serial / T_hybrid)

5T·3P·1.390 — 1,4878×
efisiensi 49,59% · 449,77 ants/s

1T·3P·1.390 — 1,4896×
efisiensi 49,65% · 443,51 ants/s

5T·2P·1.390 — 1,4060×
efisiensi 70,30% · 395,26 ants/s

5T·3P·500 — 0,9190×
efisiensi 30,63% · 231,73 ants/s

ANALISIS PERFORMA

Apa yang diukur vs apa yang masih dugaan

Sudah terukur

Personal 1.390 semut: 1,4878× (lebih cepat).

500 semut: 0,9190× (hybrid lebih lambat).

Efisiensi = speedup / jumlah proses × 100%.

Throughput = jumlah semut / waktu eksekusi.

Manfaat bergantung ukuran beban kerja.

Kemungkinan penjelas (belum terbukti)

- Serialisasi / pickling & transfer snapshot
- IPC antarproses
- Barrier menunggu worker terakhir
- Merge serial di main process
- Kontensi CPU: 3 worker pada 2 core

Biaya-biaya ini terlihat dari desain, tetapi belum diukur terpisah; perlu profiling, bukan diklaim sebagai bottleneck.

KETERBATASAN & VALIDASI

Yang belum dibuktikan oleh pengujian ini

Food collected = 0 pada semua konfigurasi benchmark.

Belum ada perjalanan lengkap makanan kembali ke sarang; benchmark ini belum membuktikan keberhasilan fungsional simulasi.

Determinisme ≠ kebenaran

Kesesuaian hasil serial dan hybrid menunjukkan konsistensi pada pengujian ini, bukan bahwa seluruh perilaku simulasi benar.

Satu kali ukur per konfigurasi

Hasil bersifat observasional; belum ada bukti signifikansi statistik.

Tanpa pengukuran biaya terpisah

Pickling, IPC, barrier, merge, transfer snapshot, dan memori belum diukur sendiri.

Satu mesin

MPI tidak diuji; tidak ada klaim eksekusi multi-node.

KESIMPULAN & REFERENSI

Manfaat hybrid bergantung ukuran beban kerja

Kesimpulan

Hybrid computing berjalan: ProcessPool (CPU) + ThreadPool (I/O).

Personal 1.390 semut dipercepat (1,4878×).

500 semut justru lebih lambat (0,9190×).

Keterbatasan: food=0; sekali ukur; tanpa profiling biaya.

Pekerjaan lanjutan

1. Selidiki penyebab food collected = 0.

2. Tambah uji fungsional pergerakan & pengumpulan.

3. Ulangi benchmark untuk median & variasi.

4. Profiling sebelum optimasi lanjutan.

Referensi (IEEE)

[1] Python Software Foundation. concurrent.futures, Python 3 docs.
[2] G. M. Amdahl, Validity of the Single Processor Approach, AFIPS, 1967.
[3] Dorigo, Maniezzo & Colorni, Ant System, IEEE Trans. SMC-B, 1996. [4] Python Software Foundation. multiprocessing, Python 3 docs.